

CSC 411 – Code Improvement Through Profiling

Lab Notes

Stage 0 – Baseline

Benchmark Results:

- midmark: 0.337 seconds
- sandmark: 7.844 seconds

Relative to Start: 1.000

Relative to Previous: 1.000

Explanation:

At this stage, we ran the original Universal Machine implementation without any optimizations to establish a baseline for performance comparison. The execution time for the sandmark benchmark was 7.844 seconds.

No profiling or optimizations were applied. We expected the main execution loop to be the primary bottleneck due to repeated instruction execution and memory access.

Stage 1 – Link-Time Optimization (LTO) (KEPT)

Benchmark Results:

- sandmark: ~5.001 seconds

Relative to Start: 0.637

Relative to Previous: 0.637

Explanation:

In this stage, we enabled link-time optimization (LTO) in the release profile. This allows the compiler to optimize across module boundaries and remove unnecessary abstractions.

The SANDmark benchmark improved significantly from 7.844 seconds to about 5.001 seconds, showing a strong performance gain. This optimization was beneficial overall and was kept.

Stage 2 – Instruction Decoding Optimization (KEPT)

Benchmark Results:

- sandmark: ~4.9–5.0 seconds

Relative to Start: ~0.63

Relative to Previous: slight improvement

Explanation:

We optimized instruction decoding by replacing helper logic with direct bit manipulation to extract opcodes and register values.

This reduced overhead inside the main execution loop, which runs for every instruction. Since decoding happens extremely frequently, even a small improvement here had a noticeable effect. This optimization was kept.

Stage 3 – Inlining Instruction Logic (KEPT)

Benchmark Results:

- sandmark: ~4.7–4.9 seconds

Relative to Start: ~0.60

Relative to Previous: moderate improvement

Explanation:

Originally, some instructions were handled through separate function calls. We removed these function calls and inlined the logic directly inside the match statement.

This reduced function call overhead and improved CPU efficiency. Because this change directly impacted the hottest part of the program, it resulted in better performance and was kept.

Stage 4 – Memory Handling Optimization (KEPT)

Benchmark Results:

- sandmark: ~4.5–4.7 seconds

Relative to Start: ~0.57

Relative to Previous: noticeable improvement

Explanation:

We improved how memory was handled, especially during the load program instruction. Previously, we were cloning arrays unnecessarily, which added significant overhead.

We switched to using references whenever possible and only copying when required. This reduced memory allocations and improved runtime. This optimization was kept.

Stage 5 – Reducing Bounds Checks / Direct Access (KEPT)

Benchmark Results:

- sandmark: ~4.45–4.6 seconds

Relative to Start: ~0.56

Relative to Previous: slight improvement

Explanation:

We minimized repeated bounds checking when accessing registers and memory. Since registers are fixed in size, we ensured they were accessed directly using indexing instead of additional checks.

This reduced overhead in tight loops and contributed to a small but consistent improvement. This optimization was kept.

Final Result

After all optimizations, our best SANDmark runtime was approximately 4.45 seconds, compared to the original 7.844 seconds.

Overall, we achieved a significant performance improvement by focusing on the main execution loop, reducing unnecessary overhead, and optimizing memory usage.

These changes demonstrate how low-level optimizations and careful attention to frequently executed code can greatly improve performance in systems-level programs.